

A Synthetic Reconstruction of Multiparty Session Types [POPL'26]

David Castro-Perez
University of Kent

Francisco Ferreira
Royal Holloway

Sung-Shik Jongmans
University of Groningen

A Synthetic Reconstruction of Multiparty Session Types [POPL'26]

David Castro-Perez
University of Kent

Francisco Ferreira
Royal Holloway

Sung-Shik Jongmans
University of Groningen

Long(er)-term research aim:

To develop new **methods** and **tools** to make
concurrent programming easier (theory $\cap$ practice)

Long(er)-term research aim:

To develop new **methods** and **tools** to make
concurrent programming easier (theory $\cap$ practice)

This paper:

What:

Communication
through
message passing

Why:

Reasoning about
safety and **liveness**
is hard

How:

Static verification
via **type checking**

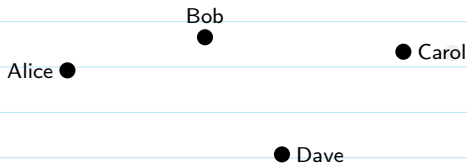
Suppose we have a **specification** S of...

1. ???

2. ???

3. ???

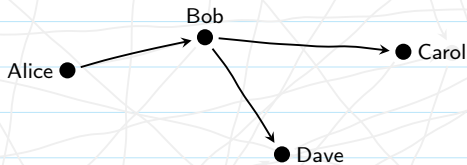
...that **implementation** I should consist of



Suppose we have a **specification** S of...

1. processes
2. ???
3. ???

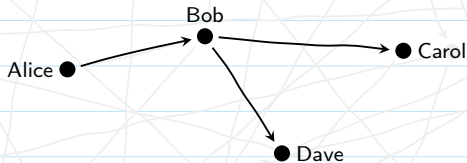
...that **implementation** I should consist of



Suppose we have a **specification** S of...

1. **processes**
2. **channels**
3. ???

...that **implementation** I should consist of

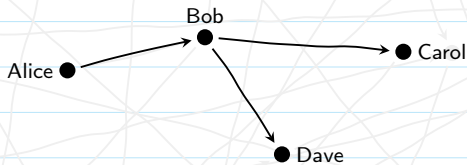


First, a number
from Alice to Bob.
Next, a boolean
from Bob to
Carol and Dave.
Next, ...

Suppose we have a **specification** S of...

1. **processes**
2. **channels**
3. **protocols** (message passing)

...that **implementation** I should consist of



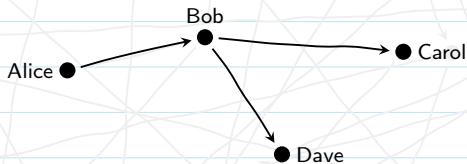
First, a number
from Alice to Bob.

Next, a boolean
from Bob to
Carol and Dave.

Next, ...

How to “prove” I is **safe** and **live** relative to S ...?

(“bad”/“good” communications never/eventually happen)



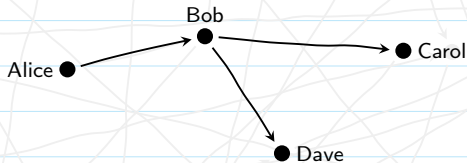
First, a number
from Alice to Bob.
Next, a boolean
from Bob to
Carol and Dave.
Next, ...

How to “prove” I is **safe** and **live** relative to S ...?

Behavioural typing:

well-typedness $\implies$ safety $\wedge$ liveness

type



First, a number
from Alice to Bob.
Next, a boolean
from Bob to
Carol and Dave.
Next, ...

How to “prove” I is **safe** and **live** relative to S ...?

B
well-type
g:
liveness

**Multiparty session
typing (MPST)**

[Honda et al., POPL'08]

type

First, a number
from Alice to Bob.

Next, a boolean
from Bob to
Carol and Dave.

Next, ...

First, a number
from Alice to Bob.

Next, a boolean
from Bob to
Carol and Dave.

Next, ...

implement as terms

Alice:

$B!(5); \text{end}$

Bob:

$A?(x:\text{Nat});$
 $[C, D]!(x == 6); \dots$

Carol:

$B?(y:\text{Bool}); \dots$

Dave:

$B?(z:\text{Bool}); \dots$

Protocol:

$$A \rightarrow B : \text{Nat} ; B \rightarrow [C, D] : \text{Bool} ; \dots$$

↑

specify as type

First, a number
from Alice to Bob.

Next, a boolean
from Bob to
Carol and Dave.

Next, ...

implement as terms

Alice:

$B!(5) ; \text{end}$

Bob:

$A?(x:\text{Nat}) ;$
 $[C, D]!(x == 6) ; \dots$

Carol:

$B?(y:\text{Bool}) ; \dots$

Dave:

$B?(z:\text{Bool}) ; \dots$

Protocol:

$$A \rightarrow B : \text{Nat} ; B \rightarrow [C, D] : \text{Bool} ; \dots$$

???

First, a number
from Alice to Bob.

Next, a boolean
from Bob to
Carol and Dave.

Next, ...

Alice:

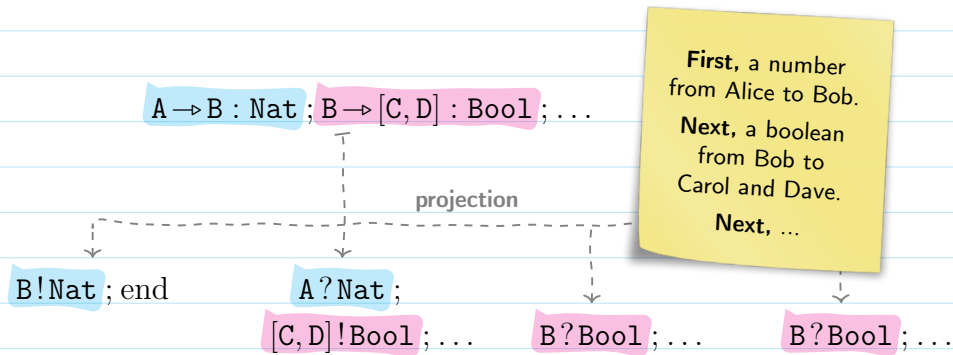
$$B!(5) ; \text{end}$$
Bob:

$$A?(x:\text{Nat}) ;$$

$$[C, D]!(x == 6) ; \dots$$
Carol:

$$B?(y:\text{Bool}) ; \dots$$
Dave:

$$B?(z:\text{Bool}) ; \dots$$



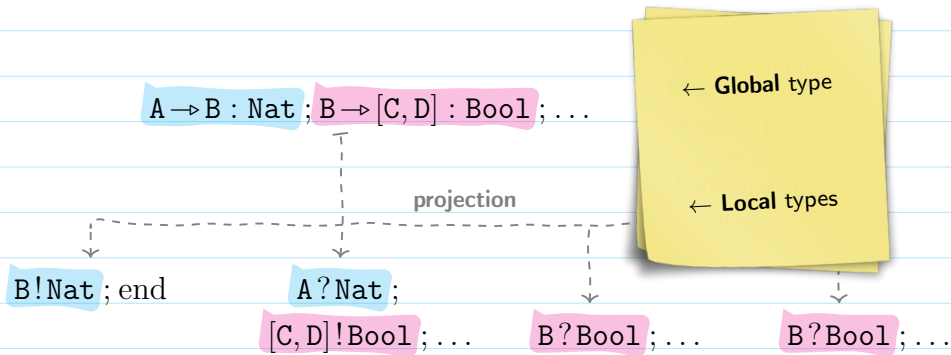
$B!(5) ; \text{end}$

$A?(x:\text{Nat}) ;$

$B?(y:\text{Bool}) ; \dots$

$B?(z:\text{Bool}) ; \dots$

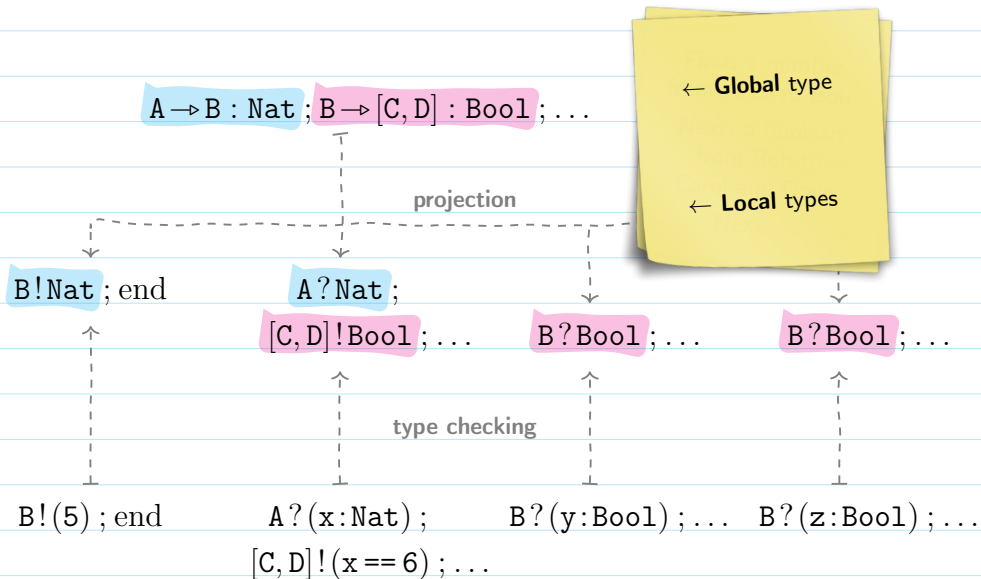
$[C, D]!(x == 6) ; \dots$

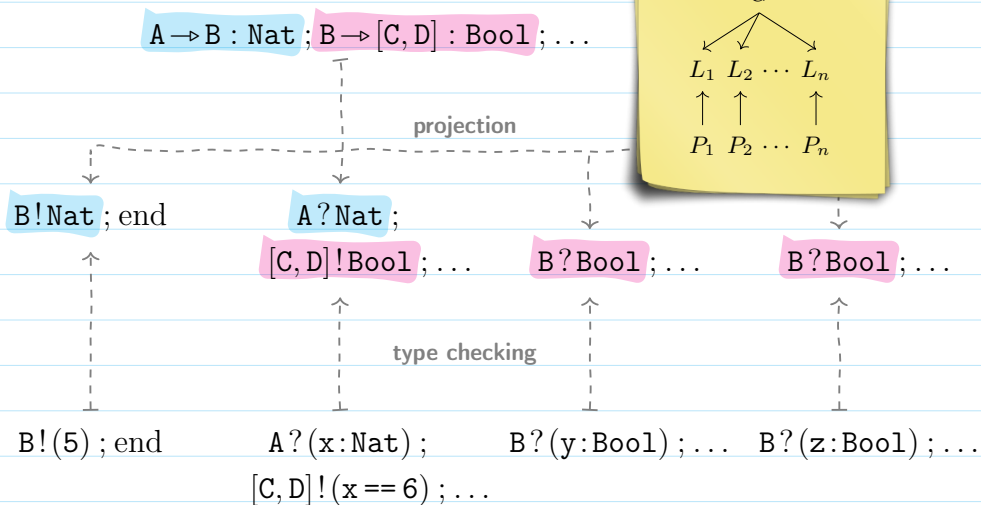


$B ! (5) ; \text{end}$

$A ? (x : \text{Nat}) ;$
 $[C, D] ! (x == 6) ; \dots$

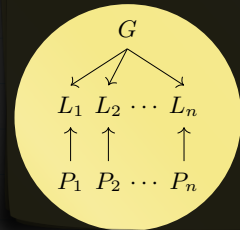
$B ? (y : \text{Bool}) ; \dots$ $B ? (z : \text{Bool}) ; \dots$





$$A \rightarrow B : \text{Nat} ; B \rightarrow [C, D] : \text{Bool} ; \dots$$

projection



Theorem: (progress)

$$B ! \text{Nat} ; \text{end}$$

$$A ? \text{Nat} ;$$

$$[C, D] ! \text{Bool} ; \dots$$

$$B ? \text{Bool} ; \dots$$

$$B ? \text{Bool} ; \dots$$

$$\begin{array}{c}
 \varphi(L_1, \dots, L_n) \wedge \prod P_i \quad \text{type checking} \quad \Rightarrow \quad \exists(P'_1, \dots, P'_n, \alpha) \quad \underbrace{\prod P_i \xrightarrow{\alpha} \prod P'_i}_{\text{liveness}} \\
 \wedge \underbrace{\vdash P_1 : L_1 \wedge \dots \wedge \vdash P_n : L_n}_{\text{well-typedness}}
 \end{array}$$

$$B ! (5) ; \text{end}$$

$$A ? (x : \text{Nat}) ;$$

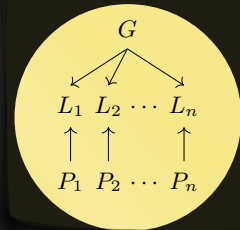
$$B ? (y : \text{Bool}) ; \dots$$

$$B ? (z : \text{Bool}) ; \dots$$

$$[C, D] ! (x == 6) ; \dots$$

$$A \rightarrow B : \text{Nat} ; B \rightarrow [C, D] : \text{Bool} ; \dots$$

projection



Theorem: (preservation)

$$B ! \text{Nat} ; \text{end}$$

$$A ? \text{Nat} ;$$

$$[C, D] ! \text{Bool} ; \dots$$

$$B ? \text{Bool} ; \dots$$

$$B ? \text{Bool} ; \dots$$

$$\varphi(L_1, \dots, L_n) \wedge \prod P_i \xrightarrow{\alpha} \prod P'_i \quad \exists(L'_1, \dots, L'_n)$$

$$\wedge \underbrace{\vdash P_1 : L_1 \wedge \dots \wedge \vdash P_n : L_n}_{\text{well-typedness}} \xRightarrow{\text{type checking}} \vdash P'_1 : L'_1 \wedge \dots \wedge \vdash P'_n : L'_n$$

$$B ! (5) ; \text{end}$$

well-typedness

$$A ? (x : \text{Nat}) ;$$

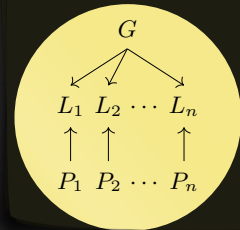
$$B ? (y : \text{Bool}) ; \dots$$

$$B ? (z : \text{Bool}) ; \dots$$

$$[C, D] ! (x == 6) ; \dots$$

$$A \rightarrow B : \text{Nat} ; B \rightarrow [C, D] : \text{Bool} ; \dots$$

projection



Theorem: (preservation)

$$B ! \text{Nat} ; \text{end}$$

$$A ? \text{Nat} ;$$

$$[C, D] ! \text{Bool} ; \dots$$

$$B ? \text{Bool} ; \dots$$

$$B ? \text{safety} ; \dots$$

$$\begin{aligned} & \varphi(L_1, \dots, L_n) \wedge \prod P_i \xrightarrow{\alpha} \prod P'_i \quad \exists(L'_1, \dots, L'_n) \quad \prod L_i \xrightarrow{\alpha} \prod L'_i \wedge \\ & \wedge \underbrace{\vdash P_1 : L_1 \wedge \dots \wedge \vdash P_n : L_n}_{\text{well-typedness}} \xRightarrow{\text{type checking}} \vdash P'_1 : L'_1 \wedge \dots \wedge \vdash P'_n : L'_n \end{aligned}$$

$$B ! (5) ; \text{end}$$

well-typedness

$$A ? (x : \text{Nat}) ;$$

$$B ? (y : \text{Bool}) ; \dots$$

$$B ? (z : \text{Bool}) ; \dots$$

$$[C, D] ! (x == 6) ; \dots$$

Assumption: $A \rightarrow B : \text{Nat} ; B \rightarrow [C, D] : \text{Bool} ; \dots$

$$L_1 = G \upharpoonright r_1 \quad \wedge \quad \dots \quad \wedge \quad L_n = G \upharpoonright r_n$$

$$\Rightarrow \varphi(L_1, \dots, L_n)$$

Theorem: (preservation)

$$\begin{aligned} & \varphi(L_1, \dots, L_n) \quad \wedge \quad \prod P_i \xrightarrow{\alpha} \prod P'_i \\ & \wedge \quad \underbrace{\vdash P_1 : L_1 \quad \wedge \quad \dots \quad \wedge \quad \vdash P_n : L_n}_{\text{well-typedness}} \quad \xRightarrow{\text{type checking}} \quad \vdash P'_1 : L'_1 \quad \wedge \quad \dots \quad \wedge \quad \vdash P'_n : L'_n \end{aligned}$$

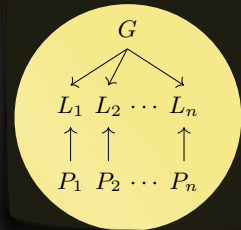
$B!(5); \text{end}$

$A?(x:\text{Nat});$

$B?(y:\text{Bool}); \dots$

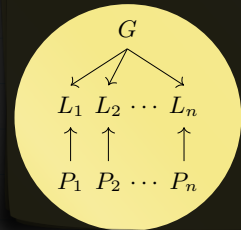
$B?(z:\text{Bool}); \dots$

$[C, D]!(x == 6); \dots$



Assumption:  $A \rightarrow B : \text{Nat} ; B \rightarrow [C, D] : \text{Bool} ; \dots$ [Scalas/Yoshida, POPL 19]

~~$L_1 = G \vdash r_1 \wedge \dots \wedge L_n = G \vdash r_n$~~ reduction
 ~~$\Rightarrow \varphi(L_1, \dots, L_n)$~~

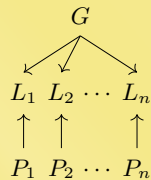


Theorem: (preservation)

$$\begin{array}{c}
 \begin{array}{c} B! \text{Nat} ; \text{end} \\ A? \text{Nat} ; \\ [C, D]! \text{Bool} ; \dots \end{array} \\
 \varphi(L_1, \dots, L_n) \wedge \prod P_i \xrightarrow{\alpha} \prod P'_i \\
 \wedge \underbrace{\vdash P_1 : L_1 \wedge \dots \wedge \vdash P_n : L_n}_{\text{well-typedness}} \\
 B!(5) ; \text{end} \quad A?(x:\text{Nat}) ; \quad [C, D]!(x == 6) ; \dots
 \end{array}
 \xRightarrow{\text{type checking}}
 \begin{array}{c}
 B? \text{Bool} ; \dots \quad \underbrace{B? \text{safety} ; \dots}_{\text{safety}} \\
 \exists(L'_1, \dots, L'_n) \prod L_i \xrightarrow{\alpha} \prod L'_i \wedge \\
 \vdash P'_1 : L'_1 \wedge \dots \wedge \vdash P'_n : L'_n \\
 B?(y:\text{Bool}) ; \dots \quad B?(z:\text{Bool}) ; \dots
 \end{array}
 \end{array}$$

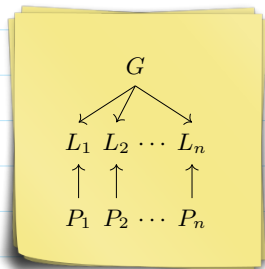
Original approach

[Honda et al., POPL'08]



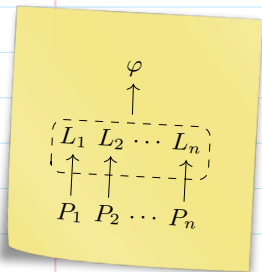
Original approach

[Honda et al., POPL'08]



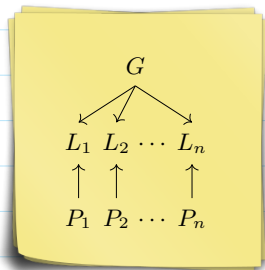
“Less Is More” approach

[Scalas/Yoshida, POPL'19]



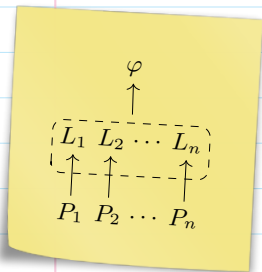
Original approach

[Honda et al., POPL'08]



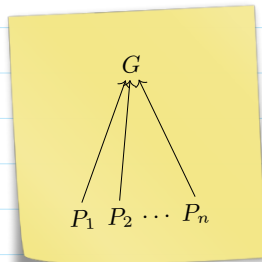
“Less Is More” approach

[Scalas/Yoshida, POPL'19]



Synthetic approach

[this paper]



Original approach

[Honda et al., POPL'08]

Compositional

“Less Is More” approach

[Scalas/Yoshida, POPL'19]

Not compositional

Synthetic approach

[this paper]

Compositional

Contributions:

- Special case: MPST system using global types (without projection)
- Special case: Pass “Less Is More” benchmark, compositionally

Contributions:

- Special case: MPST system using *global types* (without projection)
- Special case: Pass “Less Is More” benchmark, compositionally
- General case: MPST system using *well-behaved LTSs*
- General case: *Type soundness* (without φ)
- General case subsumes special case

Contributions:

- Special case: MPST system using *global types* (without projection)
- Special case: Pass “*Less Is More*” benchmark, compositionally
- General case: MPST system using *well-behaved* LTSs
- General case: *Type soundness* (without φ)
- General case subsumes special case
- Artifact: *Mechanisation* of all definitions/theorems (Agda)
- Artifact: *Implementation* of the special case (Rascal)



Contributions:

- Special case: MPST system using **global types** (without projection)
- Special case: Pass **“Less Is More”** benchmark, compositional
- General case: MPST system using ***well-behaved* LTSs**
- General case: **Type soundness** (without φ)
- General case subsumes special case
- Artifact: **Mechanisation** of all definitions/theorems (Agda)
- Artifact: **Implementation** of the special case (Rascal)

Rest of this talk:
Intuition behind technique



Problem: φ of local types isn't entailed by projection

Solution: Type check terms against global types (without φ , local types, projection)

Problem: φ of local types isn't entailed by projection

Solution: Type check terms against global types (without φ , local types, projection)

Steps:

1. ???

2. ???

3. ???

4. Prove progress/preservation:

$$\begin{aligned} \prod P_i \not\downarrow \quad \wedge \quad \vdash \prod P_i : G &\implies \exists(P'_1, \dots, P'_n, \alpha) \quad \prod P_i \xrightarrow{\alpha} \prod P'_i \\ \prod P_i \xrightarrow{\alpha} \prod P'_i \quad \wedge \quad \vdash \prod P_i : G &\implies \exists(G') \quad G \xrightarrow{\alpha} G' \quad \wedge \quad \vdash \prod P'_i : G' \end{aligned}$$

Problem: φ of local types isn't entailed by projection

Solution: Type check terms against global types (without φ , local types, projection)

Steps:

1. Define operational semantics of terms

2. ???

3. ???

4. Prove progress/preservation:

$$\begin{array}{l}
 \prod P_i \not\Downarrow \quad \wedge \quad \vdash \prod P_i : G \implies \exists (P'_1, \dots, P'_n, \alpha) \quad \prod P_i \xrightarrow{\alpha} \prod P'_i \\
 \prod P_i \xrightarrow{\alpha} \prod P'_i \quad \wedge \quad \vdash \prod P_i : G \implies \exists (G') \quad G \xrightarrow{\alpha} G' \quad \wedge \quad \vdash \prod P'_i : G'
 \end{array}$$

Problem: φ of local types isn't entailed by projection

Solution: Type check terms against global types (without φ , local types, projection)

Steps:

1. Define operational semantics of terms
2. Define operational semantics of global types
3. ???
4. Prove progress/preservation:

$$\begin{array}{l}
 \prod P_i \not\rightarrow \quad \wedge \vdash \prod P_i : G \implies \exists (P'_1, \dots, P'_n, \alpha) \quad \prod P_i \xrightarrow{\alpha} \prod P'_i \\
 \prod P_i \xrightarrow{\alpha} \prod P'_i \quad \wedge \vdash \prod P_i : G \implies \exists (G') \quad G \xrightarrow{\alpha} G' \quad \wedge \vdash \prod P'_i : G'
 \end{array}$$

Problem: φ of local types isn't entailed by projection

Solution: Type check terms against global types (without φ , local types, projection)

Steps:

1. Define operational semantics of terms
2. Define operational semantics of global types
3. Define typing rules
4. Prove progress/preservation:

$$\begin{array}{l}
 \prod P_i \not\Downarrow \quad \wedge \quad \vdash \prod P_i : G \quad \Longrightarrow \quad \exists (P'_1, \dots, P'_n, \alpha) \quad \prod P_i \xrightarrow{\alpha} \prod P'_i \\
 \prod P_i \xrightarrow{\alpha} \prod P'_i \quad \wedge \quad \vdash \prod P_i : G \quad \Longrightarrow \quad \exists (G') \quad G \xrightarrow{\alpha} G' \quad \wedge \quad \vdash \prod P'_i : G'
 \end{array}$$

Send (at p)Recv. (at q)

Comm.

Problem: φ of local types isn't entailed by projection

Solution: type check terms against global types (without φ , local types, projection)

$$\frac{e \Downarrow v}{q!e; P' \xrightarrow{pq!v} P'} \quad \frac{\vdash v:t}{p?(x:t); Q' \xrightarrow{pq?v:t} Q'[v/x]} \quad \frac{P \xrightarrow{pq!v} P' \quad Q \xrightarrow{pq?v:t} Q'}{P|Q \xrightarrow{p \rightarrow q:t} P'|Q'}$$

Steps:

1. Define operational semantics of terms
2. Define operational semantics of global types
3. Define typing rules
4. Prove progress/preservation:

$$\prod P_i \not\Downarrow \wedge \vdash \prod P_i : G \implies \exists(P'_1, \dots, P'_n, \alpha) \quad \prod P_i \xrightarrow{\alpha} \prod P'_i$$

$$\prod P_i \xrightarrow{\alpha} \prod P'_i \wedge \vdash \prod P_i : G \implies \exists(G') \quad G \xrightarrow{\alpha} G' \wedge \vdash \prod P'_i : G'$$

Send (at p)Recv. (at q)

Comm.

Problem: φ of local types isn't entailed by projection

Solution: type check terms against global types (without φ , local types, projection)

$$\frac{e \Downarrow v \quad \vdash v : t \quad P \xrightarrow{pq!v} P' \quad Q \xrightarrow{pq?v:t} Q'}{q!e ; P' \xrightarrow{pq!v} P' \quad p?(x:t) ; Q' \xrightarrow{pq?v:t} Q'[v/x] \quad P | Q \xrightarrow{p \rightarrow q:t} P' | Q'}$$

Steps:

1. Define operational semantics of terms
2. Define operational semantics of global types
3. Define typing rules
4. Prove progress/preservation:

$$p \rightarrow q:t ; G' \xrightarrow{p \rightarrow q:t} G'$$

$$\begin{aligned} \prod P_i \not\sim \quad \wedge \quad \vdash \prod P_i : G &\implies \exists (P'_1, \dots, P'_n, \alpha) \quad \prod P_i \xrightarrow{\alpha} \prod P'_i \\ \prod P_i \xrightarrow{\alpha} \prod P'_i \quad \wedge \quad \vdash \prod P_i : G &\implies \exists (G') \quad G \xrightarrow{\alpha} G' \quad \wedge \quad \vdash \prod P'_i : G' \end{aligned}$$

Send (at p)Recv. (at q)

Comm.

Problem: φ of local types isn't entailed by projection

Solution: type check terms against global types (without φ , local types, projection)

$$\frac{e \Downarrow v \quad \vdash v : t \quad P \xrightarrow{pq!v} P' \quad Q \xrightarrow{pq?v:t} Q'}{q!e ; P' \xrightarrow{pq!v} P' \quad p?(x:t) ; Q' \xrightarrow{pq?v:t} Q'[v/x] \quad P|Q \xrightarrow{p \rightarrow q:t} P'|Q'}$$

Steps:

1. Define operational semantics of terms
2. Define operational semantics of global types
3. Define typing rules
4. Prove progress/preservation:

$$p \rightarrow q:t ; G' \xrightarrow{p \rightarrow q:t} G''$$

$$\frac{\prod P_i \xrightarrow{p \rightarrow q:t} G' \quad \wedge \quad \vdash \prod P_i : G \xrightarrow{p \rightarrow q:t} G' \quad \prod P_i \xrightarrow{\alpha} \prod P'_i \quad \Gamma \vdash e : t^\alpha \quad \Gamma \vdash P' : G' \quad \vdash \prod P_i \quad \Gamma, x:t \vdash Q' : G' \quad G \xrightarrow{\alpha} G' \quad \vdash P : G \quad \vdash Q : G'}{\Gamma \vdash q!e ; P' : G \quad \Gamma \vdash p?(x:t) ; Q' : G' \quad \vdash P|Q : G}$$

Problem: φ of local types isn't entailed by projection

Solution: Type check terms against global types (without φ , local types, projection)

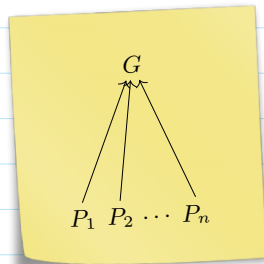
Steps:

1. Define operational semantics of terms
2. Define operational semantics of global types
3. Define typing rules
4. Prove progress/preservation:

Also in the paper:

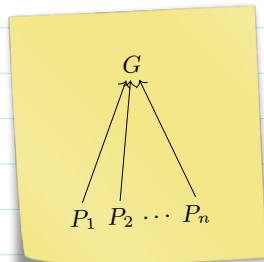
selection (easy),
if/let (easy),
"skipping" (hard),
recursion (hard)

$$\begin{array}{l}
 \prod P_i \not\vdash \quad \wedge \quad \vdash \prod P_i : G \implies \exists (P'_1, \dots, P'_n, \alpha) \quad \prod P_i \xrightarrow{\alpha} \prod P'_i \\
 \prod P_i \xrightarrow{\alpha} \prod P'_i \quad \wedge \quad \vdash \prod P_i : G \implies \exists (G') \quad G \xrightarrow{\alpha} G' \quad \wedge \quad \vdash \prod P'_i : G'
 \end{array}$$



This paper:

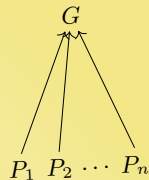
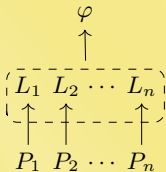
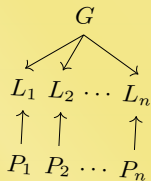
Synthetic typing rules



This paper:

Synthetic typing rules

$$\frac{G \xrightarrow{p \multimap q:t} G' \quad \Gamma \vdash e : t \quad \Gamma \vdash P' : G'}{\Gamma \vdash q!e; P' : G}$$



Original/“Less Is More”:

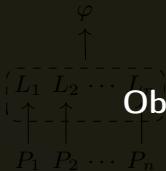
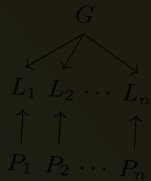
Analytic typing rules

$$\frac{\Gamma \vdash e : t \quad \Gamma \vdash P' : L'}{\Gamma \vdash q!e ; P' : q!t ; L'}$$

This paper:

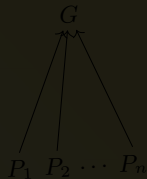
Synthetic typing rules

$$\frac{G \xrightarrow{p \rightarrow q : t} G' \quad \Gamma \vdash e : t \quad \Gamma \vdash P' : G'}{\Gamma \vdash q!e ; P' : G}$$



Observation:

???



Observation:

Original/"Less Is More":

???

Analytic typing rules

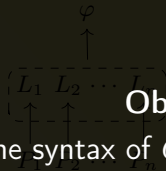
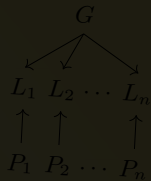
This paper:

Synthetic typing rules

$$\frac{\Gamma \vdash e : t \quad \Gamma \vdash P' : L'}{\Gamma \vdash q!e; P' : q!t; L'}$$



$$\frac{G \xrightarrow{p \rightarrow q:t} G' \quad \Gamma \vdash e : t \quad \Gamma \vdash P' : G'}{\Gamma \vdash q!e; P' : G}$$



Observation:

The syntax of G doesn't really matter



Observation:

Original/"Less Is More":

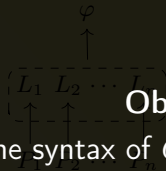
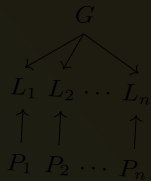
Analytic typing rules

$$\frac{\Gamma \vdash e : t \quad \Gamma \vdash P' : L'}{\Gamma \vdash q!e; P' : q!t; L'}$$

This paper:

Synthetic typing rules

$$\frac{G \xrightarrow{p \rightarrow q:t} G' \quad \Gamma \vdash e : t \quad \Gamma \vdash P' : G'}{\Gamma \vdash q!e; P' : G}$$



Observation:

The syntax of G doesn't really matter



Observation:

Original/"Loss Is Moral": G is just a state of a labelled transition system (LTS)

Analytic typing rules

Synthetic typing rules

$$\frac{\Gamma \vdash e : t \quad \Gamma \vdash P' : L'}{\Gamma \vdash q!e; P' : q!t; L'}$$



$$\frac{G \xrightarrow{p \rightarrow q:t} G' \quad \Gamma \vdash e : t \quad \Gamma \vdash P' : G'}{\Gamma \vdash q!e; P' : G}$$

Definition:

An LTS is well-behaved when it satisfies conditions 1, 2, 3, and 4.

Definition:

An LTS is well-behaved when it satisfies condition

See paper
(incl. flavours of
determinism, commu-
tativity, confluence)

Definition:

An LTS is **well-behaved** when it satisfies condition

See paper
(incl. flavours of
determinism, commu-
tativity, confluence)

Theorem: (general case)

If G is the state of a **well-behaved** LTS, then:

$$\begin{aligned} \prod P_i \not\sim \quad \wedge \quad \vdash \prod P_i : G &\implies \exists (P'_1, \dots, P'_n, \alpha) \quad \prod P_i \xrightarrow{\alpha} \prod P'_i \\ \prod P_i \xrightarrow{\alpha} \prod P'_i \quad \wedge \quad \vdash \prod P_i : G &\implies \exists (G') \quad G \xrightarrow{\alpha} G' \quad \wedge \quad \vdash \prod P'_i : G' \end{aligned}$$

Definition:

An LTS is **well-behaved** when it satisfies condition

See paper
(incl. flavours of
determinism, commu-
tativity, confluence)

Theorem: (general case)

If G is the state of a **well-behaved** LTS, then:

$$\begin{aligned} \prod P_i \not\rightarrow \quad \wedge \quad \vdash \prod P_i : G &\implies \exists (P'_1, \dots, P'_n, \alpha) \quad \prod P_i \xrightarrow{\alpha} \prod P'_i \\ \prod P_i \xrightarrow{\alpha} \prod P'_i \quad \wedge \quad \vdash \prod P_i : G &\implies \exists (G') \quad G \xrightarrow{\alpha} G' \quad \wedge \quad \vdash \prod P'_i : G' \end{aligned}$$

Theorem: (special case)

If G is a global type, then the LTS of G is **well-behaved**.

Contributions:

- Special case: MPST system using **global types** (without projection)
- Special case: Pass **“Less Is More”** benchmark, compositionality
- General case: MPST system using ***well-behaved* LTSs**
- General case: **Type soundness** (without φ)
- General case subsumes special case
- Artifact: **Mechanisation** of all definitions/theorems (Agda)
- Artifact: **Implementation** of the special case (Rascal)

Future work:

asynchrony; mixed choice; combination with model checking; ...

